

Distributed Computing Project Proposal

Pallavi Desai, Eric Nelson

Problem Statement and Background

For our final project, we will study how causal consistency mechanisms behave under frequent membership changes in distributed systems. Causal consistency is a widely used correctness model, commonly implemented using vector clocks or related metadata structures. While effective under stable membership, conventional vector-clock-based mechanisms can scale poorly in high-churn environments due to their $O(n)$ metadata size and accumulation of obsolete entries for departed nodes. Frequent joins and leaves can cause causal metadata to retain obsolete entries, inflate message sizes, and introduce delivery delays from unnecessary dependency tracking.

These issues are increasingly relevant in modern systems such as auto-scaling cloud services and geo-distributed deployments, where node membership can change rapidly in response to workload and demand. This project aims to empirically analyze how causal metadata grows and impacts performance under sustained membership churn, and to evaluate whether principled techniques, such as metadata pruning, garbage collection, or Dotted Version Vector clocks, can reduce overhead while preserving causal guarantees. If time permits, we would like to explore associating each vector entry with a lease, and automatically expire causal dependencies when the lease is not renewed, reducing stale metadata from departed nodes.

Proposed Approach

We will build a Python-based event simulation using a tool like SimPy to model a distributed key-value store with hundreds of nodes under varying churn loads. First, we will compare Dotted Version Vectors (DVV) to standard vector clocks. DVV improves upon standard vector clocks by separating discrete events, or dots, from the broader causal context, “summary vector”, which prevents metadata bloat during concurrent updates. Second, if time permits, we will develop our proposed Lease-Based DVV. This method extends DVV by assigning an expiration time to each entry in the summary vector. If a node does not heartbeat within the lease window, its entry is pruned from the summary vector via a garbage collector. This introduces time-bounded causal tracking in that after a lease expiry, we no longer guarantee tracking dependencies involving that node, and we trade completeness of causality for reduced metadata in the presence of churn. We will analyze this tradeoff of causal guarantees and churn bloat in our report.

We also hope to leverage the code here to get us started with our node implementation:

- [UCSC - Distributed kv-store](#)
- [DVV implementation](#)

Expected Contributions (Novelty)

Our contributions will come in three parts:

1. Simulation framework to benchmark different causal clocks under dynamic membership. This will include simulating stable, low churn, sustained churn, and burst churn environments.
2. Empirical analysis, comparing vector clocks, DVV, and DVV + lease:
 - a. Metadata per message
 - b. Metadata growth vs churn
 - c. Latency statistics
 - d. Dependency queue length (if relevant)
 - e. Causal violations (for lease method)
3. Implementation and comparison of a lease-based pruning for causal integrity within a fixed time window for high churn environments.

We expect to demonstrate that while DVV handles high concurrency better than traditional vector clocks, our lease extension significantly reduces the metadata footprint during high-churn periods. The final deliverable will be a validated proof-of-concept and a reusable SimPy framework for testing consistency protocols.

References

1. Preguiça, N. M., Baquero, C., Almeida, P. S., Fonte, V., & Gonçalves, R. (2010). Dotted version vectors: Logical clocks for optimistic replication. CoRR, abs/1011.5808.
2. Kulkarni, S. S., Demirbas, M., Madeppa, D., Avva, B., & Leone, M. (2014). *Logical physical clocks and consistent snapshots in globally distributed databases*. In Proceedings of the 18th International Conference on Principles of Distributed Systems (OPODIS 2014), Lecture Notes in Computer Science, vol. 8878 (pp. 17–32). Springer.